

Lab01 – Creating a Cluster with Kubeadm

In this exercise, you will learn how to create a cluster using kubeadm. The cluster will contain of a single control plane node named `kube-control-plane`, and two worker nodes named `kube-node1` and `kube-node2`.

You can find a full description of the (<https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/create-cluster-kubeadm/>) in the official Kubernetes documentation.

Initializing the Control Plane Node

1. Git clone the <https://github.com/brahimhamdi/k8s-lab> repo, and then cd to k8s-lab directory.

```
brahim@Training:~$ git clone https://github.com/brahimhamdi/k8s-lab
Clonage dans 'k8s-lab'...
remote: Enumerating objects: 29, done.
remote: Counting objects: 100% (29/29), done.
remote: Compressing objects: 100% (20/20), done.
remote: Total 29 (delta 8), reused 29 (delta 8), pack-reused 0
Réception d'objets: 100% (29/29), 10.64 Kio | 265.00 Kio/s, fait.
Résolution des deltas: 100% (8/8), fait.
brahim@Training:~$ cd k
bash: cd: k: Aucun fichier ou dossier de ce type
brahim@Training:~$ cd k8s-lab
brahim@Training:~/k8s-lab$
```

2. Deploy vagrant environment using the command `vagrant up`.

```
brahim@Training:~/k8s-lab$ vagrant up
Bringing machine 'kube-control-plane' up with 'virtualbox' provider...
Bringing machine 'kube-node1' up with 'virtualbox' provider...
Bringing machine 'kube-node2' up with 'virtualbox' provider...
=== kube-control-plane: Importing base box 'generic/ubuntu2004'...
=== kube-control-plane: Matching MAC address for NAT networking...
=== kube-control-plane: Checking if box 'generic/ubuntu2004' version '4.2.14' is up to date...
=== kube-control-plane: A newer version of the box 'generic/ubuntu2004' for provider 'virtualbox' is
=== kube-control-plane: available! You currently have version '4.2.14'. The latest is version
=== kube-control-plane: '4.2.16'. Run 'vagrant box update' to update.
=== kube-control-plane: Setting the name of the VM: k8s-lab_kube-control-plane_1689795061604_38153
=== kube-control-plane: Clearing any previously set network interfaces...
=== kube-control-plane: Preparing network interfaces based on configuration...
    kube-control-plane: Adapter 1: nat
    kube-control-plane: Adapter 2: hostonly
=== kube-control-plane: Forwarding ports...
    kube-control-plane: 22 (guest) => 2222 (host) (adapter 1)
=== kube-control-plane: Running 'pre-boot' VM customizations...
=== kube-control-plane: Booting VM...
=== kube-control-plane: Waiting for machine to boot. This may take a few minutes...
    kube-control-plane: SSH address: 127.0.0.1:2222
    kube-control-plane: SSH username: vagrant
    kube-control-plane: SSH auth method: private key
    kube-control-plane: Warning: Connection reset. Retrying...
```

...

3. Shell into control plane node using the command `vagrant ssh kube-control-plane`. Initializing the control plane using the `kubeadm init` command. Provide

`10.244.0.0/16` as the IP addresses for the Pod network. Use `192.168.56.10` for the IP address the API Server will advertise it's listening on.

```
brahim@Training:~/k8s-lab$ vagrant ssh kube-control-plane
Last login: Wed Jul 19 19:49:34 2023 from 10.0.2.2
vagrant@kube-control-plane:~$ sudo kubeadm init --pod-network-cidr 10.244.0.0/16 --apiserver-advertise-address 192.168.56.10
[init] Using Kubernetes version: v1.27.4
[preflight] Running pre-flight checks
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your internet connection
[preflight] You can also perform this action in beforehand using 'kubeadm config images pull'
W0719 20:00:02.105104 35842 checks.go:835] detected that the sandbox image "registry.k8s.io/pause:3.6" of the container runtime is inconsistent with that used by kubeadm. It is recommended that using "registry.k8s.io/pause:3.9" as the CRI sandbox image.
[certs] Using certificateDir folder "/etc/kubernetes/pki"
```

4. After the `init` command finished, run the necessary commands to run the cluster as non-root user.

```
vagrant@kube-control-plane:~$ mkdir -p $HOME/.kube
vagrant@kube-control-plane:~$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
vagrant@kube-control-plane:~$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
vagrant@kube-control-plane:~$ █
```

5. Download flannel yamli

(<https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.yml>) file and make necessary changes to use eth1 as network interface on all nodes.

```
vagrant@kube-control-plane:~$ wget https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.yml
--2023-07-19 20:06:10-- https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.yml
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.108.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.111.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4615 (4.5K) [text/plain]
Saving to: 'kube-flannel.yml'

kube-flannel.yml           100%[=====] 4.51K --.-KB/s   in 0.001s

2023-07-19 20:06:11 (7.24 MB/s) - 'kube-flannel.yml' saved [4615/4615]

vagrant@kube-control-plane:~$ vim kube-flannel.yml
vagrant@kube-control-plane:~$ █
```

```

- name: kube-flannel
  image: docker.io/flannel/flannel:v0.22.0
  #image: docker.io/rancher/mirrored-flannelcni-flannel:v0.22.0
  command:
    - /opt/bin/flanneld
  args:
    - --ip-masq
    - --kube-subnet-mgr
    - --iface=eth1
  resources:
    requests:
      cpu: "100m"
      memory: "50Mi"
```

6. Install flannel using the command `kubectl apply -f kube-flannel.yml`.

```
vagrant@kube-control-plane:~$ kubectl apply -f kube-flannel.yml
namespace/kube-flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
serviceaccount/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
vagrant@kube-control-plane:~$
vagrant@kube-control-plane:~$ █
```

7. Verify that the control plane node indicates the "Ready" status with the command `kubectl get nodes -o wide`.

```
vagrant@kubernetes-control-plane:~$ kubectl get nodes -o wide
NAME                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER
kubernetes-control-plane Ready    control-plane 11m   v1.27.4   192.168.56.10 <none>        Ubuntu 20.04.5 LTS   5.4.0-139-generic   containerd
vagrant@kubernetes-control-plane:~$
```

8. Copy `kubeadm join` command generated by `sudo kubeadm token create --print-join-command` command, and then exit out of the VM using the command `exit`.

```
vagrant@kubernetes-control-plane:~$ sudo kubeadm token create --print-join-command
kubeadm join 192.168.56.10:6443 --token 871ibd.wc0x7dqpbttrjj77a --discovery-token-ca-cert-hash sha256:8f97c9c18c7e0ad440d4a7e07536be4197ce19b7
ab004e9bdf461a51ab7e56c9
vagrant@kubernetes-control-plane:~$
vagrant@kubernetes-control-plane:~$ exit
logout
brahim@Training:~/k8s-lab$
```

Joining the Worker Nodes

9. Shell into first worker node using the command `vagrant ssh kube-node1`. Join the worker node to cluster using the `kubeadm join` command. Provide the join token and CA cert hash.

```
brahim@Training:~/k8s-lab$ vagrant ssh kube-node1
vagrant@kubernetes-node1:~$ sudo kubeadm join 192.168.56.10:6443 --token 871ibd.wc0x7dqpbttrjj77a --discovery-token-ca-cert-hash sha256:8f97c9c18c7e0
ad440d4a7e07536be4197ce19b7ab004e9bdf461a51ab7e56c9
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...

This node has joined the cluster:
* Certificate signing request was sent to apiserer and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
vagrant@kubernetes-node1:~$
```

10. Verify that the worker node indicates the "Ready" status with the command `kubectl get nodes`.

```
vagrant@kubernetes-node1:~$ exit
logout
brahim@Training:~/k8s-lab$ vagrant ssh kube-control-plane
Last login: Wed Jul 19 20:25:43 2023 from 10.0.2.2
vagrant@kubernetes-control-plane:~$ kubectl get nodes -o wide
NAME                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER
kubernetes-control-plane Ready    control-plane 26m   v1.27.4   192.168.56.10 <none>        Ubuntu 20.04.5 LTS   5.4.0-139-generic   containerd
kubernetes-node1     Ready    <none>    2m2s   v1.27.4   192.168.56.11 <none>        Ubuntu 20.04.5 LTS   5.4.0-139-generic   containerd
vagrant@kubernetes-control-plane:~$
```

11. Exit out of the VM using the command `exit`. Repeat the steps for worker node `kube-node2`. Exit out of the VM using the command `exit`.

```

vagrant@kubernetes-control-plane:~$ exit
logout
brahim@Training:~/k8s-lab$ vagrant ssh kube-node2
vagrant@kubernetes-node2:~$ sudo kubeadm join 192.168.56.10:6443 --token 871ibd.wc0x7dqpbtjrj77a --discovery-token-ca-cert-hash sha256:8f97c9c18c7e0
ad440d4a7e07536be4197ce19b7ab604e9bdf461a51ab7e56c9
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectcl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...

This node has joined the cluster:
* Certificate signing request was sent to apiser and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectcl get nodes' on the control-plane to see this node join the cluster.

vagrant@kubernetes-node2:~$
vagrant@kubernetes-node2:~$ exit
logout
brahim@Training:~/k8s-lab$
brahim@Training:~/k8s-lab$ vagrant ssh kube-control-plane
Last login: Wed Jul 19 20:26:32 2023 from 10.0.2.2
vagrant@kubernetes-control-plane:~$ kubectcl get nodes -o wide
NAME                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
kubernetes-control-plane Ready    control-plane 29m   v1.27.4   192.168.56.10 <none>        Ubuntu 20.04.5 LTS  5.4.0-139-generic   containerd
kubernetes-node1     Ready    <none>      5m10s v1.27.4   192.168.56.11 <none>        Ubuntu 20.04.5 LTS  5.4.0-139-generic   containerd
kubernetes-node2     NotReady <none>      100s   v1.27.4   192.168.56.12 <none>        Ubuntu 20.04.5 LTS  5.4.0-139-generic   containerd
vagrant@kubernetes-control-plane:~$

```

Verifying the Installation

12. Check that all nodes have been correctly registered and are in the "Ready" status.

```

vagrant@kubernetes-control-plane:~$ kubectcl get nodes -o wide
NAME                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
kubernetes-control-plane Ready    control-plane 30m   v1.27.4   192.168.56.10 <none>        Ubuntu 20.04.5 LTS  5.4.0-139-generic   containerd
kubernetes-node1     Ready    <none>      6m39s v1.27.4   192.168.56.11 <none>        Ubuntu 20.04.5 LTS  5.4.0-139-generic   containerd
kubernetes-node2     Ready    <none>      3m9s   v1.27.4   192.168.56.12 <none>        Ubuntu 20.04.5 LTS  5.4.0-139-generic   containerd
vagrant@kubernetes-control-plane:~$

```

13. Check that cluster is ok, and all system pods are running in the kube-system namespace, also the flannel pods in kube-flannel namespace.

```

vagrant@kubernetes-control-plane:~$ kubectcl cluster-info
Kubernetes control plane is running at https://192.168.56.10:6443
CoreDNS is running at https://192.168.56.10:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectcl cluster-info dump'.
vagrant@kubernetes-control-plane:~$
vagrant@kubernetes-control-plane:~$ kubectcl get pod --namespace kube-system
NAME                                READY    STATUS    RESTARTS   AGE
coredns-5d78c9869d-7s5rw            1/1     Running   0           32m
coredns-5d78c9869d-jq2z1            1/1     Running   0           32m
etcd-kubernetes-control-plane        1/1     Running   1           32m
kubernetes-apiserver-kubernetes-control-plane 1/1     Running   1           32m
kubernetes-controller-manager-kubernetes-control-plane 1/1     Running   0           32m
kubernetes-proxy-8n9tb               1/1     Running   0           32m
kubernetes-proxy-c9hrf               1/1     Running   0           8m45s
kubernetes-proxy-vlbcx               1/1     Running   0           5m15s
kubernetes-scheduler-kubernetes-control-plane 1/1     Running   1           33m
vagrant@kubernetes-control-plane:~$
vagrant@kubernetes-control-plane:~$ kubectcl get pod --namespace kube-flannel
NAME                                READY    STATUS    RESTARTS   AGE
kubernetes-flannel-ds-f8mgc          1/1     Running   0           24m
kubernetes-flannel-ds-pqcdx          1/1     Running   0           9m8s
kubernetes-flannel-ds-vphv9          1/1     Running   0           5m38s
vagrant@kubernetes-control-plane:~$
vagrant@kubernetes-control-plane:~$

```

14. Create a new Pod named `web` with the image `nginx`. Check the node the Pod has been scheduled on.

```
vagrant@k8s-control-plane:~$ kubectl get pod -o wide
No resources found in default namespace.
vagrant@k8s-control-plane:~$ kubectl run web --image=nginx
pod/web created
vagrant@k8s-control-plane:~$ kubectl get pod -o wide
NAME    READY   STATUS             RESTARTS   AGE   IP          NODE    NOMINATED NODE   READINESS GATES
web     0/1     ContainerCreating   0           14s   <none>      kube-node1   <none>           <none>
vagrant@k8s-control-plane:~$ kubectl get pod -o wide
NAME    READY   STATUS    RESTARTS   AGE   IP          NODE    NOMINATED NODE   READINESS GATES
web     1/1     Running   0           2m21s  10.244.1.2   kube-node1   <none>           <none>
```